INF-2700: Database systems

Assignment 1

Jens Christian Valen Leynse
jeley4465@uit.no

Part I. SQL

Task 1. Database schema

Describe the inf2700_orders database schema.
It is sufficient to show the CREATE TABLE statements that defined the schema.

The inf2700_order database schema consists of eight tables, each containing various attributes or columns for data management. Each attribute contains a data type (integer, text, or real) and is assigned a default value, either Null or not Null (empty or not). Most of the tables also contain a primary key as the unique identification for each row of data, and a foreign key used to establish relationships with columns in other relational tables. The table with the foreign key either references the parent table or itself through a self-reference.

```
CREATE TABLE Customers (
  customerNumber INTEGER PRIMARY KEY,
  customerName TEXT NOT NULL,
  contactLastName TEXT NOT NULL,
  contactFirstName TEXT NOT NULL,
  phone TEXT NOT NULL,
  addressLine1 TEXT NOT NULL,
  addressLine2 TEXT NULL,
  city TEXT NOT NULL,
  state TEXT NULL,
  postalCode TEXT NULL,
  country TEXT NOT NULL,
  salesRepEmployeeNumber INTEGER NULL,
  creditLimit REAL NULL
)
```

```
CREATE TABLE Employees (
  employeeNumber INTEGER PRIMARY KEY,
  lastName TEXT NOT NULL,
  firstName TEXT NOT NULL,
  extension TEXT NOT NULL,
  email TEXT NOT NULL,
  officeCode TEXT NOT NULL,
  reportsTo INTEGER NULL,
  jobTitle TEXT NOT NULL,
  FOREIGN KEY (reportsTo)
  REFERENCES Employees(employeeNumber)
)
```

```
CREATE TABLE Products (
  productCode TEXT PRIMARY KEY,
  productName TEXT NOT NULL,
  productLine TEXT NOT NULL,
  productScale TEXT NOT NULL,
  productVendor TEXT NOT NULL,
  productDescription TEXT NOT NULL,
  quantityInStock INTEGER NOT NULL,
  buyPrice REAL NOT NULL,
  MSRP REAL NOT NULL,
  FOREIGN KEY (productLine)
  REFERENCES Productlines
```

```
CREATE TABLE OrderDetails (
  orderNumber INTEGER NOT NULL,
  productCode TEXT NOT NULL,
  quantityOrdered INTEGER NOT NULL,
  priceEach REAL NOT NULL,
  orderLineNumber INTEGER NOT NULL,
  PRIMARY KEY (orderNumber, productCode),
  FOREIGN KEY (productCode)
  REFERENCES Products
)
```

```
)

CREATE TABLE Orders (
  orderNumber INTEGER PRIMARY KEY,
  orderDate TEXT NOT NULL,
  requiredDate TEXT NOT NULL,
  shippedDate TEXT NULL,
  status TEXT NOT NULL,
  comments TEXT NULL,
  customerNumber INTEGER NOT NULL,
  FOREIGN KEY (customerNumber)
  REFERENCES Customers
)

CREATE TABLE Payments (
  customerNumber INTEGER NOT NULL,
  checkNumber TEXT NOT NULL,
  paymentDate TEXT NOT NULL,
  amount REAL NOT NULL,
  PRIMARY KEY (customerNumber, checkNumber),
  FOREIGN KEY (customerNumber)
  REFERENCES Customers
)

CREATE TABLE Offices (
  officeCode TEXT PRIMARY KEY,
  city TEXT NOT NULL,
  phone TEXT NOT NULL,
  addressLine1 TEXT NOT NULL,
  addressLine2 TEXT NULL,
  state TEXT NULL,
  country TEXT NOT NULL,
  postalCode TEXT NOT NULL,
  territory TEXT NOT NULL
)

CREATE TABLE ProductLines(
  productLine TEXT PRIMARY KEY,
  description TEXT NULL
)
```

Task 2. Run the given SQL queries

Run the queries listed below.

For each query, write in your own words in a few lines that describe its purpose and how it works.

```
SELECT customerName, contactLastName, contactFirstName
FROM Customers;
```

Description: Fetches the names of all businesses and the credentials of their customer contact persons.

Implementation: The SELECT clause specifies the query output as the columns three columns: ‘customerName’, ‘contactLastName’ and ‘contactFirstName’. These columns are retrieved from the Customers table, as specified in the FROM clause.

```
SELECT *
FROM Orders
WHERE shippedDate IS NULL;
```

Description: Fetches all orders that have not yet been shipped.

Implementation: The SELECT clause specifies all columns from the orders table where the “shippedDate is set to the value NULL” condition is correct.

```
SELECT C.customerName AS Customer, SUM(OD.quantityOrdered) AS Total
```

```
FROM Orders O, Customers C, OrderDetails OD
WHERE O.customerNumber = C.customerNumber
AND O.orderNumber = OD.orderNumber
GROUP BY O.customerNumber
ORDER BY Total DESC;
```

Description: Fetches the total quantity of items ordered by each customer.
Implementation: Implementation: The SELECT clause retrieves the ‘customerName’ column and calculates the total quantity of items ordered by each customer, using the alias ‘Total'. This data is obtained from the ‘customers’ and ‘orderDetails’ tables, filtered by ‘customerNumber’ and ‘orderNumber’, connected through the orders table with WHERE clauses. The GROUP BY clause groups rows with the same ‘customerNumber’ to avoid duplication, and the ORDER BY clause arranges the results in descending order based on the ‘Total’ calculation.

```
SELECT P.productName, T.totalQuantityOrdered
FROM Products P NATURAL JOIN
        (SELECT productCode, SUM(quantityOrdered) AS totalQuantityOrdered
        FROM OrderDetails GROUP BY productCode) AS T
WHERE T.totalQuantityOrdered >= 1000;
```

Description: Fetches a list of products that have been ordered in quantities over 1,000 units.
Implementation: The SELECT clause specifies the ‘productName’ column from the ‘Products’ table and introduces a new column ‘totalQuantityOrdered.’ The FROM clause utilizes a natural join to combine the ‘Products’ table with a subquery. Within the subquery, it combines the ‘productCode’ and calculates the sum of ‘quantityOrdered’ from the ‘OrderDetails’ table. This sum is also given the alias ‘totalQuantityOrdered.’ The subquery groups the products by ‘productCode,’ and the result set is aliased as ‘T’. This alias ‘T’ is then used in the WHERE clause to filter the results, including only products that have been ordered in quantities greater than 1,000.

Task 3. Write your own SQL queries
All of the problems have been solved in the assignment1_queries.sql file in the delivery folder.

Part II. Database programming

Task: Implement a SQL tester

Introduction: The goal of this task was to ensure the proper functioning of tests in the SQL tester to generate a desired result. To achieve this, modifications were made to the “prepare test,” “query execute,” and “terminate test” functions. In sequence, the “prepare test” was updated to being able to open the test database, and in case of failure, return a value of ‘0’. The “query execute” function was updated to run the query, store the results in a dedicated variable and similarly to the preparation function, , and also return ’0’, only in this case with an error message included. Finally, the “terminate test” function was updated to close the test database for efficient resource management.

Discussion: In the preparation function the initial approach was to search for the “testdb_file_name” value already present in the function. However, it was found to be more

convenient to directly input the relational path. In the execution function, the initial idea was to use the “execute” function from the sqlite library, as it seemed suitable for the task. However, it was quickly discovered that “execute” didn’t handle multiple queries, so it was replaced by the “prepare” and “step” functions. Additionally, the “step” loop required additional formatting to be accepted by the test function as it didn’t display the output in a way the tests would recognise. Lastly, there are two “if” tests for the query execution in addition to the loop. However, it was observed that the second test to determine if the query is anything but complete didn't trigger in the execution.

Conclusion: Throughout this assignment I gained great insight into SQL queries and their practical implementation. Describing the SQL queries in task two provided a great situation for practicing the terminology. Crafting the SQL queries for the third task presented a challenge that progressively increased in complexity, making it a valuable learning experience. And adapting the functions to handle the query tests offered insights as to what to keep in mind when setting up query tests and how to properly manage queries. Althought there were a couple of delays, the assignment was completed in a timely manner with satisfactory results.